Interrogez vos données avec MongoDB

Le langage de MongoDB repose sur Javascript, ce qui facilite l'utilisation de JSon, des appels de fonctions, la création de librairies ou la définition de programmes Map/Reduce.

Toute commande sur la collection restaurants utilise le préfixe : "**db.restaurants**".

Il suffira d'y associer la fonction souhaitée pour avoir un résultat.

Avant de commencer, il faut voir à quoi ressemble un document de notre collection de restaurants. Utilisons la fonction "*findOne()*".

```
Unset

db.restaurants.findOne()





{

   "_id" : ObjectId("594b9172c96c61e672dcd689"),

   "restaurant_id" : "30075445",

   "name" : "Morris Park Bake Shop",

   "borough" : "Bronx",

   "cuisine" : "Bakery",
```

```
   "address" : {

      "building" : "1007",

      "coord" :{"type":"Point","coordinates":[-73.856077,40.848447]},

      "street" : "Morris Park Ave",

      "zipcode" : "10462"

   },

   "grades" : [

      {"date" : ISODate("2014-03-03T00:00:00.000Z"),"grade" : "A","score" :
2},

      {"date" : ISODate("2013-09-11T00:00:00.000Z"),"grade" : "A","score" :
6},

      {"date" : ISODate("2013-01-24T00:00:00.000Z"),"grade" : "A","score" :
10},

      {"date" : ISODate("2011-11-23T00:00:00.000Z"),"grade" : "A","score" :
9},

      {"date" : ISODate("2011-03-10T00:00:00.000Z"),"grade" : "B","score" :
14}
```

```
    ]

}
```

Chaque restaurant a un nom, un quartier (*borough*), le type de cuisine, une adresse (document imbriqué, avec coordonnées GPS, rue et code postale), et un ensemble de notes (résultats des inspections).

Filtrer et Projeter les données avec un motif JSon

Le point fondamental à retenir pour le langage de MongoDB est le "clé/valeur", avec des définitions de motifs en documents **JSON**. Ainsi, chaque requête sera un document, avec des clés et des valeurs. Le deuxième point est la structure d'un document. Chaque document est traité indépendamment, comme en relationnel, mais les documents contiennent maintenant des imbrications et des listes, ce qui n'est pas habituel. Nous allons en illustrer les conséquences à partir d'exemples.

Filtrage

Commençons par un type simple de requête : le filtrage. Nous allons pour cela utiliser la fonction "find" à appliquer directement à la collection. L'exemple ci-dessous récupère tous les restaurants dans le quartier (borough) de Brooklyn.

```
Unset

db.restaurants.find( { "borough" : "Brooklyn" } )
```

6 085 restaurants sont retournés.

Pour compter, il suffit d'ajouter la fonction "count"

```
Unset

db.restaurants.find( { "borough" : "Brooklyn" } ).count()
```

En effet, nous sommes en *Javascript* : le résultat d'un *find* est une liste dont on peut compter les documents avec la méthode de liste "count"

Maintenant, nous cherchons parmi ces restaurants ceux qui font de la cuisine italienne. Pour combiner deux "clés/valeurs", il suffit de faire le document motif donnant quels paires clés/valeurs sont recherchés :

```
Unset

db.restaurants.find(

   { "borough" : "Brooklyn",

    "cuisine" : "Italian" }

)
```

Ensuite, on va chercher les restaurants présents sur la 5° Avenue. La clé "street" est imbriquée dans l'adresse ; comment filtrer une clé imbriquée (obtenue après fusion) ? Il faut utiliser les deux clés, en utilisant un point "." comme si c'était un objet.

```
Unset

db.restaurants.find(
```

```
   { "borough" : "Brooklyn",

    "cuisine" : "Italian",

    "address.street" : "5 Avenue" }

)
```

Pourquoi ne pas chercher le mot "pizza" dans le nom du restaurant ? Pour cela, il faut utiliser les expressions régulières avec "/xxx/i" (le i pour "Insensible à la casse" - majuscule/minuscule).

Unset

```
db.restaurants.find(

   { "borough" : "Brooklyn",

    "cuisine" : "Italian",

    "address.street" : "5 Avenue",

    "name" : /pizza/i }

)
```

Il ne reste maintenant que 2 restaurants Italiens dans le quartier de Brooklyn dans la 5° Avenue, dont le nom contient le mot "pizza".

Projection

Et si nous ne gardions dans le résultat que le nom ? C'est ce que l'on appelle une projection. Un deuxième paramètre (optionnel) de la fonction *find* permet de choisir les clés à retourner dans le résultat. Pour cela, il faut mettre la clé, et la valeur "1" pour projeter la valeur (on reste dans le concept "clé/valeur").

Unset

```
db.getCollection('restaurants').find(
   {"borough":"Brooklyn",
   "cuisine":"Italian",
   "name":/pizza/i,
   "address.street" : "5 Avenue"},
   {"name":1}
)
```

```
{"_id" : ObjectId("594b9173c96c61e672dd074b"), "name" : "Joe'S Pizza"}

{"_id" : ObjectId("594b9173c96c61e672dd1c16"), "name" : "Gina'S Pizzaeria/
Deli"}
```

L'identifiant du document "_id" est automatiquement projeté, si vous souhaitez le faire disparaître il suffit de rajouter la valeur "0" : {"name":1, "_id":0}

Et si nous regardions les résultats des inspections des commissions d'hygiène ? Il faut pour cela regarder la valeur de la clé "*grades.score*".

Unset

```
db.getCollection('restaurants').find(

   {"borough":"Brooklyn",

   "cuisine":"Italian",

   "name":/pizza/i,

   "address.street" : "5 Avenue"},

   {"name" : 1,

   "grades.score" : 1}

)
```

```
{
  "name" : "Joe'S Pizza",
  "grades" : [
    {"score" : 12},
    {"score" : 13},
    {"score" : 42},
    {"score" : 25},
    {"score" : 19},
    {"score" : 40},
    {"score" : 22}
  ]
}
```

```
{
  "name" : "Gina'S Pizzaeria/ Deli",
  "grades" : [
     {"score" : 12},
     {"score" : 23},
     {"score" : 13}
  ]
}
```

Ce qui est intéressant est de voir que le score est retrouvé dans les documents imbriqués de la liste "grades", et que tous les scores sont projetés.

À noter qu'un score bas est un bon score pour une inspection aux États-Unis

Filtrage avec des opérations

Le filtrage par valeur exacte n'est pas suffisant pour exprimer tout ce que l'on souhaiterait trouver dans la collection. Je cherche maintenant les noms et scores des restaurants de Manhattan ayant un score inférieur à 10. On peut utiliser des opérateurs arithmétiques sur les clés (valeur numérique). Pour cela, il sera préfixé d'un dollar **$**. "Plus petit que" en anglais se dit "*less than*", l'opérateur est donc **$lt**. Il

faut y associer la valeur souhaitée : **"grades.score" : {"$lt" : 10}**. Vous trouverez les détails ici.

L'opérateur est **toujours** imbriqué dans un document. Le concept "clé/valeur" doit être respecté. Voici les opérateurs disponibles avec des exemples associés :

$gt, $gte	>, ≥	Plus grand que (*greater than*)	"a" : {"$gt" : 10}
$lt, $lte	<, ≤	Plus petit que (*less than*)	"a" : {"$lt" : 10}
$ne	≠	Différent de (*not equal*)	"a" : {"$ne" : 10}
$in, $nin	∈, ∉	Fait parti de (ou ne doit pas)	"a" : {"$in" : [10, 12, 15, 18] }
$or	□	OU logique	"a" : {“$or” : [{"$gt" : 10}, {“$lt” : 5}] }
$and	□	ET logique	"a" : {“$and” : [{"$lt" : 10}, {“$gt” : 5}] }
$not	¬	Négation	“a" : {“$not” : {"$lt" : 10} }
$exists	∃	La clé existe dans le document	“a” : {“$exists” : 1}
$size		test sur la taille d'une liste (uniquement par égalité)	“a” : {“$size” : 5}

La requête donne ceci :

```
Unset
```

```
db.getCollection('restaurants').find(

   {"borough":"Manhattan",

   "grades.score":{$lt : 10}

   },

   {"name":1,"grades.score":1, "_id":0})



{

   "name" : "Dj Reynolds Pub And Restaurant"

   "grades" : [

      {"score" : 2},

      {"score" : 11},

      {"score" : 12},

      {"score" : 12}
```

```
    ]

}
```

Ce qui est déroutant dans ce résultat, c'est le fait que l'on trouve des scores supérieurs à 10 !

Nous ne sommes plus en relationnel, ainsi, l'opération **"grades.score" : {"$lt" : 10}**, veut dire :

Est-ce que la liste "grades" contient un score (**au moins**) avec une valeur inférieure à 10 ?

Et en effet, il y a un score à "2" respectant la question.

Si l'on souhaite ne récupérer que ceux qui n'ont pas de score supérieur à 10, il faut alors combiner la première opération avec une négation de la condition "

≥10

≥10 ". La condition est alors vérifiée sur chaque élément de la liste.

```
Unset

db.getCollection('restaurants').find(

   {"borough":"Manhattan",

   "grades.score":{

      $lt:10,
```

```
      $not:{$gte:10}

   }

   },

   {"name":1,"grades.score":1, "_id":0})
```

```
{

   "name" : "1 East 66Th Street Kitchen",

   "grades" : [

      {"score" : 3},

      {"score" : 4},

      {"score" : 6},

      {"score" : 0}

   ]
```

```
}
```

On pourrait même corser la chose en cherchant les restaurants qui ont un grade ‘C’ avec un score inférieur à 40.

```Unset
db.restaurants.find({
   "grades.grade" : "C",
   "grades.score" : {$lt : 30}
  },
  {"Grades.grade":1, "grades.score":1}
);

{
   "_id" : ObjectId("594b9172c96c61e672dcd695"),
```

```
  "grades" : [

    {"grade" : "B","score" : 21},

    {"grade" : "A","score" : 7},

    {"grade" : "C","score" : 56},

    {"grade" : "B","score" : 27},

    {"grade" : "B","score" : 27}

  ]

}

{

  "_id" : ObjectId("594b9172c96c61e672dcd6bc"),

  "grades" : [

    {"grade" : "A","score" : 9},

    {"grade" : "A","score" : 10},
```

```
    {"grade" : "A","score" : 9},

    {"grade" : "C","score" : 32}

  ]

}
```

Le résultat est pour le moins étrange concernant le premier document car il y a bien un grade C, mais avec un score de 56 ! Si l'on regarde la requête de plus près, ce n'est pas étonnant. Y a-t-il un grade 'C' ? oui. Y a-t-il un score inférieur à 40 ? oui… Nous avons oublié de préciser qu'il fallait que ce soit vérifié sur chaque élément ! La vérification se fait sur l'intérieur de la liste, pas sur chaque élément de la liste. Pour cela, un opérateur permet de faire une vérification instance par instance : **$elemMatch**.

```
Unset

db.restaurants.find({

  "grades" : {

      $elemMatch : {

         "grade" : "C",

         "score" : {$lt :40}

      }
```

```
    }

 },

 {"grades.grade" : 1,"grades.score" : 1}

);



{

   "_id" : ObjectId("594b9172c96c61e672dcd6bc"),

   "grades" : [

        {"grade" : "A","score" : 9},

        {"grade" : "A","score" : 10},

        {"grade" : "A","score" : 9},

        {"grade" : "C","score" : 32}

   ]
```

```
}
```

Cette fois-ci le résultat est bon.

Une dernière pour finir, je voudrais maintenant les noms et quartiers des restaurants dont la dernière inspection (la plus récente, donc la première de la liste) a donné un grade ‘C’. Il faut donc chercher dans le premier élément de la liste. Pour cela il est possible de rajouter l’indice recherché (indice 0) dans la clé.

```Unset
db.restaurants.find({

   "grades.0.grade":"C"

 },

 {"name":1, "borough":1, "_id":0}

);



{"borough" : "Queens","name" : "Mcdonald'S"}

{"borough" : "Queens","name" : "Nueva Villa China Restaurant"}
```

```
{"borough" : "Queens","name" : "Tequilla Sunrise"}

{"borough" : "Manhattan","name" : "Dinastia China"}
```

Distinct

Au fait, quels sont les différents quartiers de New York ? Pour cela, on peut utiliser la fonction "distinct()" avec la clé recherchée pour avoir une liste de valeur :

```Unset
db.restaurants.distinct("borough")

[
  "Bronx",
  "Brooklyn",
  "Manhattan",
  "Queens",
  "Staten Island",
  "Missing"
```

```
]
```

Et si nous le faisions sur une liste de valeurs comme les grades donnés par les inspecteurs ?

```
Unset
db.restaurants.distinct("grades.grade");
[
  "A",
  "B",
  "Z",
  "C",
  "P",
  "Not Yet Graded"
```

```
]
```

La fonction distincte va donc chercher les instances de la liste pour en extraire chaque grade et produire une liste distincte (plutôt que de faire une liste de liste distincte).

Créer une séquence d'opérations avec "*aggregate*"

Et si maintenant, nous regardions des opérations plus complexes pour bien manipuler nos données ? Pour cela, la fonction "**aggregate ()**" permet de spécifier des chaînes d'opérations, appelées pipeline d'agrégation.

Cette fonction aggregate prend une liste d'opérateurs en paramètre. Il existe plusieurs types d'opérateurs de manipulation de données. Nous allons nous concentrer par la suite sur les principaux :

- **{$match : {} }** : C'est le plus simple, il correspond au premier paramètre de la requête find que nous avons fait jusqu'ici. Il permet donc de filtrer le contenu d'une collection.
- **{$project : {} }** : C'est le second paramètre du find. Il donne le format de sortie des documents (projection). Il peut par ailleurs être utilisé pour changer le format d'origine.
- **{$sort : {} }** : Trier le résultat final sur les valeurs d'une clé choisi.
- **{$group : {} }** : C'est l'opération d'agrégation. Il va permettre de grouper les documents par valeur, et appliquer des fonctions d'agrégat. La sortie est une nouvelle collection avec les résultats de l'agrégation.
- **{$unwind : {} }** : Cet opérateur prend une liste de valeur et produit pour chaque élément de la liste un nouveau document en sortie avec cet élément. Il pourrait correspondre à une jointure, à ceci près que celle-ci ne filtre pas les données d'origine, juste un complément qui est imbriqué dans le document. On pourrait le comparer à une jointure externe avec imbrication et listes.

Vous aurez remarqué que chaque opérateur est imbriqué dans un document, et la valeur est elle-même un autre document. C'est la structure imposée pour la définition de l'opérateur. Pour illustrer un aggregate, je vais reprendre notre dernière requête *find* :

```
Unset

db.restaurants.aggregate( [

   { $match : {

      "grades.0.grade":"C"

   }},

   { $project : {

      "name":1, "borough":1, "_id":0

   }}

] )
```

Le résultat est identique, toutefois, c’est un peu plus lourd à écrire. On identifie bien chaque opérateur, avec leur définition.

Rappelez-vous que l'interpréteur mongodb utilise le langage Javascript. Chaque opérateur peut alors être défini par une variable que nous pouvons utiliser directement :

```
Unset

varMatch = { $match : { "grades.0.grade":"C"} };
```

```
varProject = { $project : {"name":1, "borough":1, "_id":0}};
```

```
db.restaurants.aggregate( [ varMatch, varProject ] );
```

Nous utiliserons ces variables par la suite pour alléger le code en ré-utilisant des variables.

Tri

Trions maintenant le résultat par nom de restaurant par ordre croissant (croissant : 1, décroissant : -1) :

```
Unset
varSort = { $sort : {"name":1} };
db.restaurants.aggregate( [ varMatch, varProject, varSort ] );
```

Groupement simple

Comptons maintenant le nombre de ces restaurants (premier rang ayant pour valeur C). Pour cela, il faut définir un opérateur $group. Celui-ci doit contenir obligatoirement une clé de groupement (_id), puis une clé (total) à laquelle on associe la fonction d'agrégation ($sum) :

```
Unset

varGroup = { $group : {"_id" : null, "total" : {$sum : 1} } };

db.restaurants.aggregate( [ varMatch, varGroup ] );

{"_id" : null, "total" : 220}
```

Cette requête est équivalente à :

```
Unset

db.restaurants.count({"grades.0.grade":"C"})

db.restaurants.find({"grades.0.grade":"C"}).count()
```

Mais si vous souhaitez faire un pipeline d’agrégation, vous serez obligé d’utilisé la fonction “aggregate”.

Ici, pas de valeur de groupement demandé (on compte simplement). Nous avons choisi de mettre la valeur *null*, on aurait pu mettre "toto", cela fonctionne également. La clé "total" est créée dans le résultat et va effectuer la "somme des 1" pour chaque

document source. Faire une somme de 1 est équivalent à compter le nombre d'éléments.

Groupement par valeur

Comptons maintenant par quartier le nombre de ces restaurants. Il faut dans ce cas changer la clé de groupement en y mettant la valeur de la clé "borough". Mais si l'on essaye ceci :

```
Unset

varGroup2 = { $group : {"_id" : "borough", "total" : {$sum : 1} } };

db.restaurants.aggregate( [ varMatch, varGroup2 ] );

{"_id" : "borough", "total" : 220}
```

La valeur de la clé de groupement prend une unique valeur "*borough*". Le problème vient du fait que nous n'avons pas pris la valeur, mais seulement précisé un texte ! Pour prendre la valeur, il faut préfixer la clé par un dollar "$borough", cela va, lors de l'exécution, indiquer qu'il faut utiliser la valeur de la clé pour l'agrégation.

```
Unset

varGroup3 = { $group : {"_id" : "$borough", "total" : {$sum : 1} } };

db.restaurants.aggregate( [ varMatch, varGroup3 ] );

{"_id" : "Bronx",        "total" : 27.0}
```

```
{"_id" : "Staten Island",    "total" : 7.0}

{"_id" : "Manhattan",        "total" : 83.0}

{"_id" : "Brooklyn",        "total" : 56.0}

{"_id" : "Queens",        "total" : 47.0}
```

Ce qui fait bien un total de 220 !

Unwind

Maintenant, trouvons le score moyen des restaurants par quartiers, et trions par score décroissant. Pour cela, nous groupons les valeurs par quartier (*$borough*), nous utilisons la fonction d'agrégat "*$avg*" pour la moyenne (*average*), puis nous trions sur la nouvelle clé produite "$moyenne".

Un problème se pose comment calculer la moyenne sur une "*grades.score*" qui est une liste ? Il faudrait au préalable retirer chaque élément de la liste, puis faire la moyenne sur l'ensemble des valeurs. C'est l'opérateur **$unwind** qui s'en charge, il suffit de lui donner la clé contenant la liste à désimbriquer.

```
Unset
varUnwind = {$unwind : "$grades"}

varGroup4 = { $group : {"_id" : "$borough", "moyenne" : {$avg :
"$grades.score"} } };

varSort2 = { $sort : { "moyenne" : -1 } }
```

```
db.restaurants.aggregate( [ varUnwind, varGroup4, varSort2 ] );
```

```
{ "_id" : "Queens", "moyenne" : 11.634865110930088 }

{ "_id" : "Brooklyn", "moyenne" : 11.447723132969035 }

{ "_id" : "Manhattan", "moyenne" : 11.41823125728344 }

{ "_id" : "Staten Island", "moyenne" : 11.370957711442786 }

{ "_id" : "Bronx", "moyenne" : 11.036186099942562 }

{ "_id" : "Missing", "moyenne" : 9.632911392405063 }
```

Ce qu'il est important de noter dans cette fonction *aggregate*, c'est que les opérateurs sont composés en séquences : chaque opérateur prend la collection produite par l'opérateur précédent (et non la collection de départ). Ceci permet de créer de longues chaînes d'opérateurs pour faire des calculs lourds.

Attention ! L'ordre des opérations est très important (projection après les filtrages, tri sur le résultat d'un groupe, filtrage avant ou après *unwind*/*group*…).

Pour des calculs plus compliqués ne pouvant être réalisés avec un *aggregate* (structure conditionnelle, multi-output/fusion, calculs complexes…), MongoDB offre la possibilité de programmer en Map/Reduce (avec javascript). Comme cela sort du

cadre de cours sur le langage d'interrogation, vous pourrez trouver la documentation ici et un TP de mise en pratique ici.

Sachez par ailleurs que tous les opérateurs sont programmés en *Map/Reduce* pour faire l'évaluation. Le langage de MongoDB est un langage de plus haut-niveau permettant de faire l'interrogation.

Mettre à jour les données

Une base de données ne serait pas utile si nous ne pouvions faire des mises à jour sur les données. Nous avons déjà vu l'insertion (*save*) dans le chapitre précédent, il faut maintenant regarder les mises à jour (*update*) et les suppressions (*delete*). À chaque modification, il faut préciser le document ciblé (*_id*) et la modification (pour les mises à jour des documents existants).

Update

Pour commencer, nous allons ajouter un commentaire sur un restaurant (opération **$set**) :

Unset

```
db.restaurants.update (
  {"_id" : ObjectId("594b9172c96c61e672dcd689")},
  {$set : {"comment" : "My new comment"}}
);

WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

On peut voir qu'un document a été trouvé (*nMatched*) et modifié (*nModified*). Vous pouvez vérifier s'il a été modifié avec une requête *find*.

Pour supprimer une clé, il suffit de remplacer par l'opérateur **$unset**.

```
Unset

db.restaurants.update (

  {"_id" : ObjectId("594b9172c96c61e672dcd689")},

  {$unset : {"comment" : 1}}

);
```

Faisons une requête avec filtrage pour choisir les restaurants à commenter (faire les documents un par un est fastidieux). Nous allons attribuer un commentaire en fonction des *grades* obtenus. S'il n'a pas eu de note 'C', nous rajouterons le commentaire 'acceptable'.

```
Unset

db.restaurants.update (

  {"grades.grade" : {$not : {$eq : "C"}}},

  {$set : {"comment" : "acceptable"}}

);



WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Le résultat est identique. Il n'y aurait qu'un seul restaurant sans note 'C' ? Faites la requête find correspondante, vous verrez qu'il y en a 22 649 ! MongoDB interdit par défaut les modifications multiples avec un filtre. Cette opération pourrait être lourde et ralentir le système. Du coup, il faut explicitement dire que les modifications peuvent être multiples :

Unset

```
db.restaurants.update (

  {"grades.grade" : {$not : {$eq : "C"}}},

  {$set : {"comment" : "acceptable"}},

  {"multi" : true}

);
```

Update et javascript

Et si l'on cherchait à donner une note en fonction de l'ensemble des inspections effectuées par restaurant ? Cela devient compliqué avec l'opération **$set** (prend au mieux la valeur d'une clé avec un "**$**"). On va pour cela programmer un peu en *javascript* avec des fonctions itératives, et sauvegarder le résultat.

> Attention, cela peut être long car il faut exécuter la fonction pour chaque élément de la base, le modifier puis le sauvegarder.

Nous allons ajouter 3 points pour un rang A, 1 point pour B, et -1 pour C.

Unset

```
db.restaurants.find( {"grades.grade" : {$not : {$eq : "C"}}} ).forEach(

   function(restaurant){

      total = 0;

      for(i=0 ; i<restaurant.grades.length ; i++){

         if(restaurant.grades[i].grade == "A")         total += 3;

         else if(restaurant.grades[i].grade == "B")    total += 1;

         else                                          total -= 1;

      }

      restaurant.note = total;

      db.restaurants.save(restaurant);

   }

);
```

Le résultat ci-dessous montre que la meilleure note est obtenue par le "*Taco Veloz*" avec 24 points.

```
Unset

db.restaurants.find({},
{"name":1,"_id":0,"note":1,"borough":1}).sort({"note":-1});

{ "borough" : "Queens", "name" : "Taco Veloz", "note" : 24 }

{ "borough" : "Manhattan", "name" : "Gemini Diner", "note" : 22 }

{ "borough" : "Manhattan", "name" : "Au Za'Atar", "note" : 22 }

{ "borough" : "Brooklyn", "name" : "Lucky 11 Bakery", "note" : 22 }

{ "borough" : "Queens", "name" : "Mcdonald'S", "note" : 21 }

{ "borough" : "Queens", "name" : "Rincon Salvadoreno Restaurant", "note" :
21 }

{ "borough" : "Manhattan", "name" : "Kelley & Ping", "note" : 21 }
```

Remove

Nous allons maintenant supprimer tous les restaurants qui ont une note de 0.

```
Unset

db.restaurants.remove(

  {"note":0},

  {"multi" : true}

);
```

Save

Pour finir, une insertion (en dehors des possibilités d'importation avec mongoimport) s'effectue avec la fonction "*save*" :

```
Unset

db.restaurants.save({"_id" : 1, "test" : 1});
```

Dans le cas où un document de la collection aurait le même identifiant "_id", le serait automatiquement écrasé par la nouvelle valeur.